

Lab4实验报告

思考题

Thinking 4.1

保存现场的SAVE_ALL函数只改写了k0,k1,v0寄存器，这三个寄存器不会破坏通用寄存器。

可以，陷入内核调用过程没有改变a0-a3的值，可以直接调用。

调用sys开头函数前我们先把参数加载到了sys开头函数认为的位置。

Trapframe中cp0_epc增加了4，并将执行系统调用后的返回值存入了\$v0寄存器，使得系统调用后用户可以获得返回值并执行下一条指令。

Thinking 4.2

用户程序中会使用srcva为0的调用来表示只传value值而不需要传递物理页面，且envid2env中如果id=0则返回当前进程curenv，因此mkenvid不会返回0

Thinking 4.3

说明子进程和父进程共享代码且具有相同状态

说明子进程在创建时保存了父进程的上下文和PC值等

Thinking 4.4

C

Thinking 4.5

UTOPI以上空间用户不可访问，无需保护。

UXSTACKTOP - BY2PG到UXSTACKTOP的空间是用户进程的异常栈，若进行写时复制保护可能陷入死循环，因此不能被保护。

USTACKTOP到USTACKTOP + BY2PG的空间在空间分布图上是Invalid memory，用不到所以无需保护。

故需要保护的是UTEXT与USTACKTOP之间且被使用的页面。

Thinking 4.6

vpt存放着二级页表，vpd存放着一级页表，`(*vpd)[va >> 22]` 为二级页表的物理地址，`(*vpt)[va >> 12]` 为va对应的物理页面

mmu.h和entry.S中有如下代码

```
extern volatile Pte* vpt[];
extern volatile Pde* vpd[];
```

```
.globl vpt
vpt:
    .word UVPT

.globl vpd
vpd:
    .word (UVPT+(UVPT>>12)*4)
```

vpt指向UVPT区域，也就是当前进程页表项所在的位置，变量类型为Pte*数组。

vpd指向 $(UVPT + (UVPT \gg 12) * 4)$ 区域，是自映射机制下的一级页表地址，变量类型是Pde*数组。

用户进程没有权限修改页表项

Thinking 4.7

用户发生鞋时复制引发的缺页中断并进行处理时，可能会再次发生页中断，需要中断重入

因为在用户进程处理缺页中断，需要读取Trapframe，且进程在中断结束恢复需要trapframe的数据

Thinking 4.8

用户处理可以减轻内核负担，符合微内核系统理念

首先使用存放函数调用返回值的v0, v1恢复非通用寄存器，之后通过v1恢复非通用寄存器，之后通过sp恢复通用寄存器，最后恢复sp，保证了恢复后寄存器值正确

Thinking 4.9

syscall_env_allocy也可能需要异常处理

此时进程给__pgfault_handler变量赋值时就会触发缺页中断，但中断处理没有设置好，故无法进行正常处理

不需要，子进程复制了父进程中__pgfault_handler变量值

练习题

Exe 4.1

函数流程：保存用户态寄存器到栈中->找到系统调用入口->传参（注意前四个参数也要在栈中保留空间，但不用填入数值）->跳转->返回，向用户态传递返回值

Exe 4.4

第一次写完之后一直test1.1过不了，检查了好久之后发现提示中有这么一句话

```

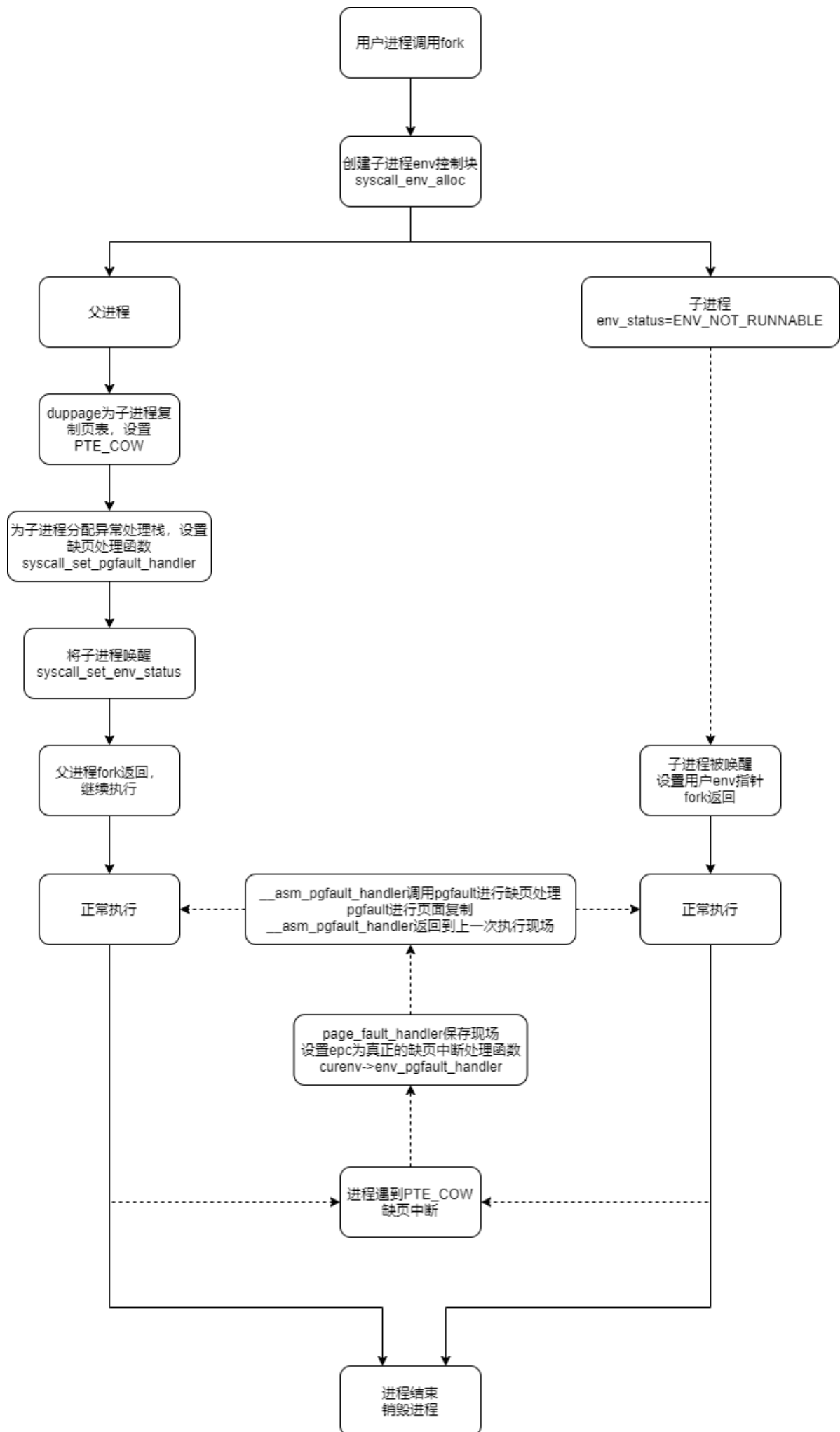
/* Overview:
 * Map the page of memory at 'srcva' in srcid's address space
 * at 'dstva' in dstid's address space with permission 'perm'.
 * Perm has the same restrictions as in sys_mem_alloc.
 * (Probably we should add a restriction that you can't go from
 * non-writable to writable?)
 *
 * Post-Condition:
 * Return 0 on success, < 0 on error.
 *
 * Note:
 * Cannot access pages above UTOP.
 */
/**** exercise 4.4 ****/
int sys_mem_map(int sysno, u_int srcid, u_int srcva, u_int dstid, u_int ds
tva,

```

因此((*pppte & PTE_R) == 0) && ((perm & PTE_R) != 0)时也要返回异常

Fork

fork函数能够从一个进程生成另一个进程，使得子进程拥有和旧进程绝大部分相同的信息。但fork会在父子进程中拥有不同的返回值。fork前代码只有父进程执行，之后代码父子进程都执行，但两者返回值不一样，子进程返回0，父进程不为0。



体会与感想

本次实验难度比lab3又更难了一个等级，花费的时间也比之前又增加了很多。lab4非常综合，串起了前三个lab的所有内容，前面的bug到lab4开始出现，非常头疼。课下改了许多之后lab4-1的课上发现还有课下lab3的bug，非常痛苦。而且发现前面的有些内容已经开始忘了，导致lab4很多内容无法串联起来，还得先返回去学之前的内容。